Alex Tefft

Notes: The majority of content added is after my project 1 section. I have marked which sections are from which projects for now.

Project 1:

(Exaggerated royalty-free stock image of the common portrayal of a computer scientist.)

In popular culture, the computer scientist is often portrayed alone in a dark room typing away at a computer console. Within the actual discipline, this often can't be further from the truth. Computer Science is more than just the study of machines and computers, it is a community of practice where writing functions as the connections between humans and computers. Whether through the syntax of a Java file or the words in a peer-reviewed paper, writing in CS is a dynamic, multimodal act that establishes professional legitimacy and mediates the progress of science itself.

As I explored through interviews with an instructor and a peer, the transition into this field involves a shift in identity. One must move away from a "problem-solving"

mindset learned in school (what my instructor interviewee called the "gold star" approach, for the “gold star” stickers teachers may give out) toward a communication mindset, where the value of a solution is determined by how effectively it can be understood, critiqued, and built upon by others. By examining these perspectives alongside threshold concepts, this paper will illustrate how writing in Computer Science reflects shared values and supports the collective participation necessary for the field to evolve.

If writing is a “dynamic, multimodal technology for making meaning,” as the threshold concept suggests, then Computer Science is perhaps the ultimate example of this definition. However, what actually counts as “writing” depends on who you ask within the community. For the peer I interviewed, the definition is broad and functional: "code is definitely writing to me." Through this view, Python or C++ are simply languages with a unique audience: the computer itself. My instructor, however, observed a divergence within the field’s genres. He noted that while a software package or block of code provides the functional implementation of a solution, the academic paper is a separate layer that links to the code rather than being a translation of it. This is an important note that suggests writing in CS is not a single act, but a multimodal stack. At the bottom layer, code mediates the activity between programming and machine, while at the top, scholarly papers mediate the activity between researchers and the greater scientific community. To be a practitioner of Computer Science is to be a translator between all of these modes, ensuring both that the machine executes the logic and the human community understands its significance.

While the medium of writing may shift, the underlying values of the discipline

seem to be focused on precision and transparency. In Computer Science, these are more than just stylistic preferences, and are instead fully functional. My peer interviewee emphasized that in code, precision is vital. If the code isn't logically sound, the "audience" (computer) will fail to respond in the way the programmer is looking for. However, this value extends to the social interactions of the field as well. My peer talked about their time working at an internship and noted that comments on his pull requests (essentially formal requests to merge new code into a main project) initially felt angry or personal. He soon realized that the bluntness was a reflection of the community's values. The goal is "functioning code," over the protection of the writer's ego. This aligns with Carolyn Miller's argument that genre is "centered not on the substance or the form of discourse but on the action it is used to accomplish" (Miller 151). In this case, the genre of the "Code Review" serves the social action of maintaining/expanding system functionality, which encourages a transparent, egoless communication style.

The transition from a student to a professional in Computer Science is marked by a shift in how one defines 'success' in writing. My instructor described this as moving away from the 'gold star' approach of k-12 and early college, where the goal is simply to solve the problem to get credit. This mirrors the findings of Johnson and Krase, who observe that the graduating seniors often point to "the importance of junior- and senior-level tasks to their development as writers" (2). They suggest that for a student to truly "come to learn" as a writer, they must move beyond a "paper-for-grade economy" (10). As my instructor noted, "you can solve a problem, but it won't matter if you can't communicate that you have solved it." This realization marks the point where a student stops "simplifying a writing task" to make it accessible for a grade and begins to realize

their own ideas as part of a "coherent field of diverse voices" (Johnson and Krase 2, 14). This illustrates the threshold concept that writing enacts disciplinarity. The act of writing the 'story' of the code is the act of becoming a scientist. By moving away from the 'gold star' mindset, the student begins to use writing to expand professional disciplinary knowledge rather than just satisfying a rubric. Writing in a discipline is a way of “coming to learn.” The writing does not just report the result, but also constructs the significance of the result. Writing moves from a passive “I found this” to an active, often persuasive argument for why the discovery matters to the wider scientific ecosystem.

If writing in Computer Science includes writing the “story” of a solution, then that story is frequently designed to provoke a specific response. This is related to the threshold concept that writing mediates activity. My instructor interviewee noted that he enjoys a more “aggressive” writing style, especially when relating to a new software package. He describes this genre as often inherently persuasive. Writers are essentially saying “My package is better; here is why you should use it.” This is also an illustration of Miller’s “Genre as Social Action.” In this context, a software writeup or a research paper is more than a neutral technical description, it is a social tool used to convince a community to adopt a new algorithm, shift their methodology, or “act in some manner” to advance the field. Writing mediates the activity between the creator’s discovery and the community’s adoption. Without a rhetorical ‘push’, even the most useful code would remain stationary and never cross from being a private project to a public contribution.

Finally, writing serves as a representation of identity, where the quality of the writing or code directly constructs the author’s professional persona. In computer science, your writing is your proxy. It is the primary way you establish what my instructor

calls “legitimacy and competence.” He pointed out that the consumer of your writing uses the clarity or precision of your writing to judge the quality of your mind. “Your writing defines you,” my instructor noted. This creates a high-pressure environment for the writer. My peer agreed with this sentiment, and admitted he felt a sense of anxiety when writing for a mathematical modeling class. He was worried that if he was perceived as a “bad communicator,” then others would be led to believe they didn’t understand the underlying math, even if the calculations were perfect. This reveals something about the discipline: to be seen as a legitimate computer scientist, one must be able to write themselves into that role, proving that identity as a computer scientist is linked to the CS documents we produce.

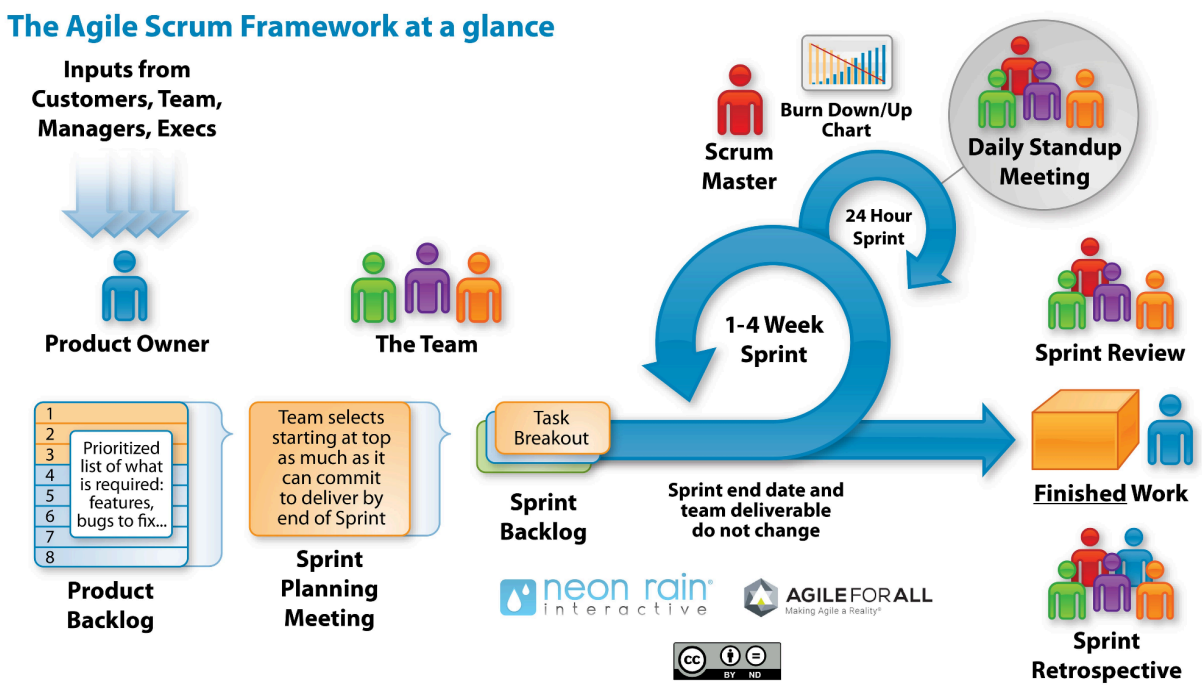
It has become clear to me that Computer Science is not the silent, purely logical practice it is often perceived to be. Instead it is a community where writing in many forms, whether via code, documentation, or scientific papers, serves as the primary vehicle for participation and progress. Writing is the tool that transforms a solution into science.

This project has started to reshape my understanding of my own identity within the field. Up until now, I have also had the “gold star” approach where most of my code is meant to comply with a strict rubric and metrics for a grade. By better understanding the values and ideologies that govern computer science writing now, I am more prepared to enter into the discipline through writing. Ultimately, I have learned that to be a successful computer scientist, one must

master the ‘full stack’ of computer science writing as much as being a master of code syntax.

Project 2:

Understanding the general values of Computer Science is only the first step to becoming a Computer Scientist. To move from the ‘gold star’ reward system of the classroom to a more professional real life working environment, it is important to be able to navigate specific sub-disciplines that each have their own rigid rules of engagement. By focusing on the specific discourse community of a Scrum team, we can see how writing functions not just as a reflection of the author’s identity, but is also a vital mediating tool for creating software.



“Scrum” is a project management framework that divides work into short iterations called “sprints,” usually lasting a few weeks. Unlike traditional methods where all of the requirements for the code are set from the beginning, Scrum acknowledges

that software requirements will emerge as development continues. This necessitates constant written communication between the people involved. I am a newcomer to this community, although I have experienced Scrum through group projects in my Software Engineering classes, so I am still learning exactly how to navigate all of the rigid and specific rituals of Scrum teams. By analyzing this community, I am looking to explore the following questions: How does writing mediate the collaborative nature of a Scrum team?

To understand how a Scrum team operates as a community, we can view it through the lens that Swales presents in “The Concept of Discourse Community” (1990) and his concept of the discourse community. Swales notes that such communities have a broadly agreed upon set of goals and methods of communication between members. For a scrum team, the primary goal is the delivery of a working software that follows all requirements from the product owner.

The mechanisms of communication employed by a Scrum team are often digital and follow specific formal formats. This community uses specialized genres that include the “User Story,” the “Sprint Backlog,” “Acceptance Criteria” and more. Each genre serves a very specific purpose. A User Story, for example, is typically always written in the format "As a [type of user], I want [some goal] so that [some reason]." The purpose of these messages is not to fully detail the implementation of a feature, but to facilitate a conversation between a business and a developer. The community employs a variety of highly specialized terms like “story points” “velocity” “standup meeting” and more. Without an understanding of these genres and lexis, a developer would simply not be able to participate in the sprint.

Finally, Swales notes that a discourse community also must have “a threshold level of members with a suitable degree of relevant content and discoursal expertise.” In a scrum team, one must both possess the technical ability to code as well as the ability to participate in and navigate Scrum’s specific communication styles. Even if a developer is a coding genius, they will remain an outsider as long as they are not following the formats of communication laid out by Scrum.

James Paul Gee’s framework allows us to examine the underlying dynamics of language, identity, and power that affect the discourse. Gee argues that entering a discourse involves more than just skill (both technical and discoursal). It requires adopting a specific identity or a “way of being in the world” that signals membership to a community. In a Scrum team, individual identity is often secondary to the “development team” identity that developers take on, where the group acts as a single unit. This ‘way of being’ is regulated by a power structure that dictates that the Product Owner defines the “what”, including the vision and requirements, while the team defines the “how”, or the technical implementation.

If a member of a Scrum team uses language focused on themselves or assigning blame, like “I finished my part, but the backend doesn’t work because of [name]” The developer will be seen as an outsider. To be seen as a legitimate member is to move to a more egoless tone where the success of the team is put over individual achievement.

To expand on the power dynamics within the Scrum Discourse, it is important to look at how the Scrum team is really structured. In a typical corporate discourse, there

is often a very straightforward top-down power structure. A manager will assign a task and a subordinate will complete the task. Scrum replaces this with 3 roles: The Product Owner, the Scrum Master, and the Team. Within this framework, no single person has total control.

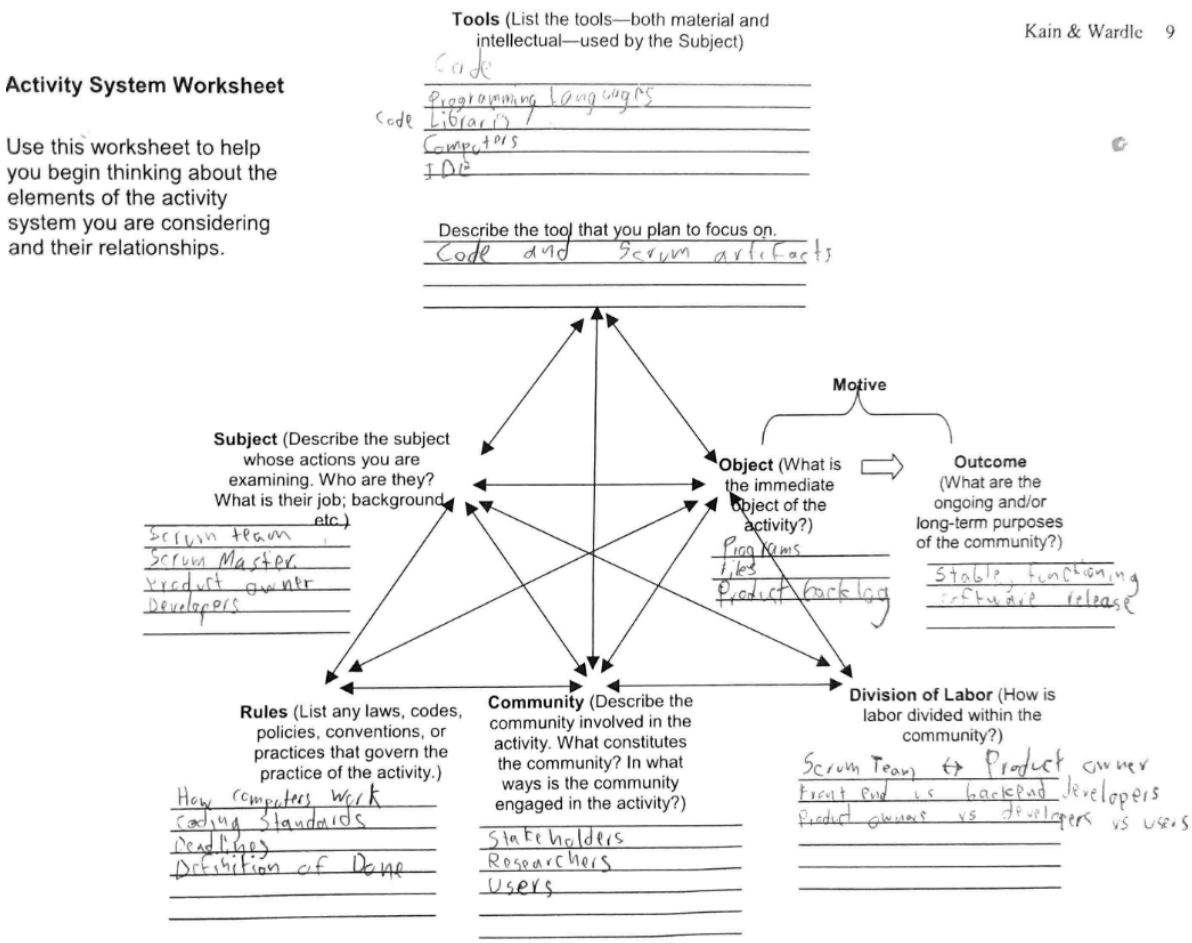
The Product Owner has the power of prioritization. They are allowed to decide what features should be included, and can order the Product Backlog (list of all tasks that need to be completed), but they do not get to decide how long it will take the team to build or how they should build it.

The development team is the party with the power to actually implement features. During a Spring Planning meeting, the team pulls items from the Product Backlog (what needs to be done) to the Sprint Backlog (what is being done this Sprint). This means the development team is the one that decides which work they will get done each sprint.

If the Product Owner controls the “what” and the development team controls the “how,” then the Scrum Master’s job is ensuring the team is adhering to all of the rituals of Scrum. They have no formal power to hire, fire, or assign tasks, and instead they are in charge of the “rules of Scrum.” Looking through Gee’s lens, the Scrum Master is the ultimate insider, and ensures all members are performing their identities “correctly.” Their primary task is often the removal of “blockers” (impediments to the developers, which are documented daily). The Scrum Master is the one who asks questions like “What is stopping us from meeting this Sprint Goal.” The Scrum Master’s power is mostly ensuring the Scrum framework stays intact and functioning.

While Gee’s framework helps us understand the identities members take on, it is through the lens of Kain & Wardle’s Activity Theory that we see how these identities

translate to products being made. In a Scrum Team, writing and language are not just used to communicate information, but are also the primary tools for facilitating activity.



In this activity system, the Subjects (Product Owner, Developers, etc) use specific tools to achieve an objective. These tools include material objects like IDEs and compilers, but more importantly, they include discursive tools like programming languages and Scrum specific genres. The ultimate outcome is not just ‘code’, but a stable, functioning software release that will provide value to the end users. This activity system is also mediated by rules and the division of labor. The rules include everything from the rules of computers (their cpu architecture, speed, limitations) to socially constructed

standards like coding standards, and an agreed upon “Definition of Done.” Meanwhile, the division of labor ensures the product owner is focusing on the “what” while the developers control the “how.” When we view Scrum as an activity system like this, it is clear that writing is the primary tool that is used to move software development forward.

By utilizing the frameworks of Swales’ Gee’ and Activity Theory, it is clear that competence in this field is not just about knowing how to code. It is about navigating a complex discourse community where your ability to communicate is just as important as the code you create. As a newcomer entering the field, I realize in order to become a great computer scientist, I must focus on mastering both the code, theory, and logic of computer science as well as the discourse communities that form it.

Works Cited

Johnson, J. Paul, and Ethan Krase. "Coming to Learn: From First-Year Composition to Writing in the Disciplines." Across the Disciplines, vol. 9, no. 2, 2012.

Miller, Carolyn R. "Genre as Social Action." Quarterly Journal of Speech, vol. 70, no. 2, 1984, pp. 151-67.

Naming What We Know: Threshold Concepts in Writing Studies. Edited by Linda Adler-Kassner and Elizabeth Wardle, Utah State UP, 2015.

Gee, James Paul. "Literacy, Discourse, and Linguistics: Introduction." *Journal of Education*, vol. 171, no. 1, 1989, pp. 5-17.

Kain, Donna, and Elizabeth Wardle. "Activity Theory: An Introduction for the Writing Classroom." *Writing About Writing: A College Reader*, edited by Elizabeth Wardle and Doug Downs, Bedford/St. Martin's, 2011, pp. 273-83.

Swales, John. "The Concept of Discourse Community." *Genre Analysis: English in Academic and Research Settings*, Cambridge UP, 1990, pp. 21-32.